

Agente RAG para DNI Valencia

Informe de la practica de Inteligencia Artificial

Arquitectura, decisiones, pruebas, benchmark y conclusiones

Integrantes: Nerea Aguilar Fores y Judit Espinoza Cervera

Estado: documentacion preparada para entrega; Banda 8 pendiente de incorporar si existen resultados RAGAs externos al repositorio.

1. Introduccion

El objetivo de la practica ha sido construir un agente RAG capaz de responder preguntas sobre la asociacion DNI Valencia a partir del corpus oficial proporcionado. El sistema combina recuperacion de informacion y generacion de lenguaje natural para ofrecer respuestas fundamentadas, con cita de fuentes y control anti-alucinacion.

La solucion entregada sigue el contrato de la practica mediante la funcion `consultar(pregunta: str, conversation_id: str | None = None) -> dict`, situada en la raiz del repositorio. La salida incluye respuesta, fuentes, chunks recuperados, metricas y trazas.

2. Arquitectura elegida

Se ha elegido una arquitectura single-agent modular. No se implementa arquitectura hexagonal completa, porque el alcance final se ha centrado en consolidar las bandas 5, 6 y 7 y documentar correctamente el sistema.

La logica esta separada en modulos:

- `chunker.py`: carga y division del corpus.
- `embedder.py`: generacion de embeddings.
- `retriever.py`: consulta a ChromaDB.
- `prompts.py`: prompt anti-alucinacion.
- `generator.py`: llamada al modelo generador.
- `pipeline.py`: orquestacion del flujo RAG.

Esta estructura mantiene el codigo comprensible y facilita futuras ampliaciones, aunque no llega al desacoplamiento completo de ports y adapters requerido para Banda 10.

3. Flujo de una consulta

El flujo completo es el siguiente:

- 1 El usuario o corrector llama a `consultar.py`.
- 2 `pipeline.answer` recibe la pregunta.
- 3 `retriever.retrieve` transforma la pregunta en embedding y recupera chunks desde ChromaDB.
- 4 `prompts.build_prompt` construye el prompt con el contexto recuperado y reglas de seguridad.
- 5 `generator.generate` invoca el modelo configurado en Ollama o PoliGPT.
- 6 El pipeline devuelve un diccionario con respuesta, fuentes, chunks y metricas.

La respuesta final debe estar basada en los documentos recuperados. Si la informacion no aparece en el contexto, el prompt obliga al modelo a responder: No tengo esa informacion en mis fuentes.

4. Corpus y chunking

El corpus contiene 16 ficheros `.txt` sobre DNI Valencia. Incluye informacion general, desayunos solidarios, RESIS, COLES, horarios, contacto, preguntas frecuentes e impacto social.

Se usa `RecursiveCharacterTextSplitter` con:

- `chunk_size=500`.
- `chunk_overlap=100`.

Esta configuracion permite conservar contexto suficiente dentro de cada fragmento y mantener prompts razonables. Cada chunk conserva el nombre del archivo origen, lo que permite citar fuentes en la salida.

5. Decisiones tomadas

ChromaDB persistente

Se usa ChromaDB persistente para evitar recalcul ar embeddings en cada ejecucion. Esto mejora la velocidad en consultas repetidas y facilita la defensa oral.

Prompt anti-alucinacion

El prompt se reforzo durante las pruebas para:

- responder solo con informacion del contexto;
- rechazar preguntas fuera de ambito;
- citar siempre el archivo fuente;
- no inventar datos numericos;
- no inventar significados de siglas;
- separar conceptos cuando la pregunta sea comparativa.

Query expansion

Se incorporo una expansion sencilla para preguntas que contienen RESIS o COLES:

- RESIS se asocia a residencias, mayores y abuelitos.
- COLES se asocia a refuerzo escolar, nin os y colegios.

Esto mejora el retrieval en preguntas comparativas.

Benchmark con varios modelos

Para Banda 7 se compararon cuatro modelos manteniendo el mismo corpus, chunking, retrieval y prompt. Asi se comparan modelos, no sistemas distintos.

6. Cambios realizados durante las pruebas

Las pruebas documentadas en pruebas/ muestran un proceso iterativo:

- Se corrigio un prompt inicial que todavia arrastraba el dominio de la plantilla.
- Se reforzo el comportamiento anti-alucinacion.
- Se aumento el numero de chunks recuperados para mejorar preguntas multi-documento.
- Se anadio expansion de consulta para RESIS y COLES.
- Se ajusto el prompt para evitar generalizaciones.
- Se anadieron reglas para no inventar significados de siglas.
- Se mejoro el formato de comparativas.
- Se ejecutaron benchmarks con modelos locales y PoliGPT.

Estos cambios mejoraron especialmente las respuestas sobre RESIS y COLES, que al principio tendian a mezclar conceptos o inventar significados.

7. Benchmark

El benchmark se realizo con 8 preguntas:

- preguntas generales sobre DNI;
- preguntas sobre desayunos solidarios;
- comparativas entre RESIS y COLES;
- preguntas sobre refuerzo escolar;
- preguntas sobre residencias;
- preguntas fuera de ambito.

Modelos evaluados:

Modelo	Entorno	Resultado cualitativo
llama3.2:3b	Ollama local	Rapido y funcional, pero limitado en sintesis compleja.
qwen2.5:3b	Ollama local	Mejor equilibrio local entre calidad, velocidad y espanol.
poligpt	PoliGPT UPV	Respuestas mas completas y mejor estructuradas, con mayor latencia.
qwen	PoliGPT UPV	Buena sintesis, pero respuestas largas y latencia elevada.

Los resultados completos estan en `benchmark/benchmark.md` y `benchmark/benchmark.json`.

8. Resultados observados

Aspectos positivos

- El agente responde correctamente preguntas especificas cuando el retrieval encuentra contexto suficiente.
- Las preguntas fuera de ambito se rechazan con la frase esperada.
- Las comparativas entre RESIS y COLES mejoraron tras introducir reglas de prompt y expansion de consulta.
- Los modelos PoliGPT produjeron respuestas mas estructuradas.
- qwen2.5:3b fue una alternativa local equilibrada.

Problemas detectados

- La pregunta "Que es DNI?" fue rechazada o respondida de forma insuficiente en algunos modelos, aunque habia contexto parcial.
- Los modelos pequenos pueden mezclar informacion cuando la pregunta requiere sintesis multi-documento.
- Algunos modelos externos generan respuestas demasiado largas, aumentando latencia y tokens.
- El benchmark estructurado referencia archivos `runs/run_*.json` que deben incorporarse si se quieren conservar los resultados crudos.

9. RAGAs y metricas propias

El enunciado indica que para Banda 8 se deben incluir:

- `faithfulness`;
- `answer_relevancy`;
- `context_precision`;
- `context_recall`;
- dos metricas propias definidas y justificadas.

En el estado actual del repositorio no se han encontrado los archivos de resultados RAGAs ni las metricas propias. Por ese motivo, la declaracion `features.json` mantiene Banda 8 en `false`.

Si el grupo dispone de esos resultados en otro lugar, deben incorporarse en `evaluacion/` antes de la entrega y actualizar este informe.

10. Limitaciones

- La arquitectura no es hexagonal.
- No hay retrieval hibrido con BM25.
- No hay re-ranking.
- No hay memoria conversacional real.
- El sistema depende de que el indice ChromaDB se haya construido previamente.
- La calidad final depende mucho del modelo elegido.
- El control anti-alucinacion puede provocar rechazos conservadores.

11. Dificultades encontradas

- Adaptar la plantilla inicial al dominio real de DNI Valencia.
- Evitar que el modelo inventara significados para siglas como RESIS.
- Conseguir buenas respuestas comparativas con modelos pequenos.
- Mantener coherencia entre `benchmark`, `features.json` y `documentacion`.
- Comparar modelos con latencias y estilos de respuesta muy distintos.

12. Conclusiones

El sistema cumple el objetivo principal de la practica: responder preguntas sobre DNI Valencia usando un pipeline RAG con fuentes y control anti-alucinacion. La solucion es especialmente solida para Banda 7, porque incluye metricas por respuesta y `benchmark` con cuatro modelos.

La mejor opcion local observada es `qwen2.5:3b`, por su equilibrio entre calidad y velocidad. `PoliGPT` ofrece respuestas mas completas, pero con mayor latencia. La principal mejora futura seria incorporar una evaluacion RAGAs completa y, si se aspirara a la maxima nota, refactorizar a arquitectura hexagonal.

13. Archivos relevantes de la entrega

- `README.md`: instrucciones y explicacion general.

- `consultar.py`: contrato principal.
- `features.json`: bandas declaradas.
- `AI_USAGE.md`: uso de IA.
- `GRUPO.md`: integrantes y reparto.
- `benchmark/`: benchmark de modelos.
- `pruebas/`: registro de pruebas.
- `docs/`: arquitectura y contrato.
- `evaluacion/`: espacio preparado para Banda 8.